Node.js — 4 Years Experience Skill Map

Backend Developer • Architecture • Performance • Security • Interview Preparation

Target: Be able to design, build, debug, optimize and deploy a production-ready Node.js backend.

1. JavaScript — Strong

• let, const, scope; Objects and Arrays; map, filter, reduce, find

• Destructuring; Spread / Rest; Closures; Hoisting; this; Prototypes

• Event Loop; Call Stack; Microtasks / Macrotasks

• Callbacks; Promises; async/await; Promise.all; Promise.allSettled; Promise.race

• Error handling and debugging

2. Node.js Core

• Event Loop; Non-blocking I/O; Blocking vs Non-blocking; Thread Pool

• Worker Threads; Streams; Buffers

• fs, path, http, events, crypto, process

• Environment variables; CommonJS vs ESM

• Graceful shutdown

3. Express.js

• Routing; Controllers; Middleware; Custom middleware; Error middleware

• req, res, next; Params; Query; Body; HTTP methods; Status codes

• Request validation; Centralized error handling; Router structure

• API versioning; CORS; Rate limiting

4. Databases

• SQL: SELECT, INSERT, UPDATE, DELETE, WHERE, JOIN, GROUP BY, ORDER BY, LIMIT

• Transactions; Indexes; Constraints; Primary key; Foreign key

• Query optimization; Connection pooling

• MongoDB: CRUD, Aggregation, Indexes, Schema design, Populate, Transactions

5. Redis

• GET / SET; EXPIRE / TTL; INCR

• Cache; Cache-aside pattern; Session storage; Rate limiting

• Pub/Sub basics; Redis connection handling; Cache invalidation

• Cache hit, cache miss, Redis vs MySQL

6. Authentication & Security

• JWT; Access token; Refresh token; Middleware authentication

• Password hashing; bcrypt; Authorization; Role-based access control

• CORS; Helmet; Rate limiting; Input validation

• SQL Injection; XSS basics; Secrets and environment variables

7. Backend Architecture

• Routes → Middleware → Controller → Service → Repository / DB

• Separation of concerns; SOLID basics; Modular architecture

• Clean code; Reusable services; Configuration management

8. Debugging

• ECONNREFUSED; ClientClosedError; Cannot set headers after they are sent

• UnhandledPromiseRejection; undefined; TypeError

• Root-cause analysis and structured debugging

9. Performance & Optimization

• Async programming; Avoid blocking operations

• Database indexes; Query optimization; Redis caching

• Pagination; Connection pooling; Compression; Streaming

• Memory leaks; Load testing; Profiling; PM2; Clustering basics

10. Testing

• Unit testing; Integration testing

• Jest; Supertest; Mocking; Test database

11. Production / Deployment

• Git; GitHub; Environment variables

• Docker basics; PM2; Nginx; CI/CD basics; Linux basics

• Logging; Monitoring; Health checks; Graceful shutdown

12. AWS / Cloud Basics

• EC2; S3; RDS; IAM; CloudWatch; Load Balancer

• Basic Node.js deployment on cloud

13. API Design

• REST; HTTP methods; HTTP status codes

• Pagination; Filtering; Sorting; Search

• Versioning; Idempotency; Error response format

• API documentation; Swagger / OpenAPI

14. Real Project Experience

• Client → Nginx → Node/Express → JWT Middleware → Controller → Service → Redis → MySQL

• Explain why Redis is used, how JWT works, validation, error handling and API optimization

- Be prepared for Redis/database failures and high-traffic scenarios

Recommended Learning Progression

Level	Focus
1	JavaScript + Promises + Callbacks
2	Express + CRUD
3	MySQL
4	Redis
5	JWT + Middleware
6	Error Handling + Validation
7	Optimization
8	Testing
9	Docker + PM2 + Nginx
10	Production Architecture
11	4-Year Interview Questions
12	Real-world Backend Project

Interview Readiness Checklist

■ Explain Event Loop, Thread Pool and Non-blocking I/O clearly.

■ Build REST CRUD APIs with Express.

■ Use Promises, async/await, callbacks and Promise.all correctly.

■ Design JWT authentication and middleware.

■ Use MySQL transactions, indexes and connection pools.

■ Implement Redis caching and rate limiting.

■ Debug common Node.js production errors.

■ Explain API optimization decisions.

■ Write basic unit and integration tests.

■ Explain deployment with Docker, PM2 and Nginx.

■ Discuss production architecture and failure scenarios.

■ Explain one real-world backend project end-to-end.

Training principle: Practice code → debug → optimize → explain in an interview → apply in a mini-project.